File Handling

- File handling in Python is the process of managing data flow to and from files stored on secondary storage devices, enabling data persistence even after a program terminates.

- The primary function for working with files is the built-in open() function.

- As the part of programming requirement, we have to store our data permanently for future purpose. For this requirement we should go for files.
- Files are very common permanent storage areas to store our data.

Types of Files:

There are 2 types of files: -

1) Text Files: Usually we can use text files to store character data
   Eg: abc.txt

2) Binary Files: Usually we can use binary files to store binary data like images, video files, audio files etc.

Opening a File:

- Before performing any operation (like read or write) on the file, first we have to open that file. For this we should use Python's inbuilt function open()

- But at the time of open, we have to specify mode,which represents the purpose of opening file.

f = open(filename, mode)

Modes Types: -

1) **r** → open an existing file for read operation. The file pointer is positioned at the beginning of the file. If the specified file does not exist, then we will get FileNotFoundError. This is default mode.

2) **w** → open an existing file for write operation. If the file already contains some data, then it will be overridden. If the specified file is not already available, then this mode will create that file.

3) **a** → open an existing file for append operation. It won't override existing data. If the specified file is not already available, then this mode will create a new file.

4) **r+** → To read and write data into the file. The previous data in the file will not be deleted. The file pointer is placed at the beginning of the file.

5) **w+** → To write and read data. It will override existing data.

6) **a+** → To append and read data from the file. It won't override existing data.

7) **x** → To open a file in exclusive creation mode for write operation. If the file already exists, then we will get FileExistsError.

Closing a File:

After completing our operations on the file, it is highly recommended to close the file.

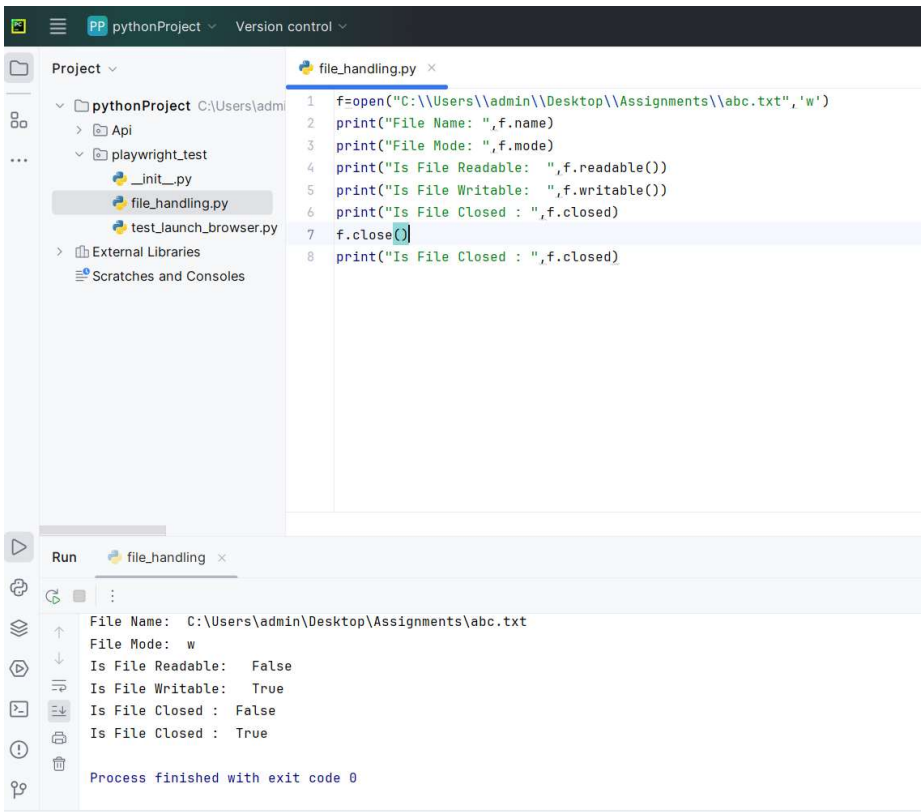
For this we have to use close() function.

f.close()

Various Properties of File Object:

Once we opened a file and we got file object, we can get various details related to that file by using its properties.

- name → Name of opened file
- mode → Mode in which the file is opened
- closed → Returns boolean value indicates that file is closed or not
- readable() → Retruns boolean value indicates that whether file is readable or not
- writable() → Returns boolean value indicates that whether file is writable or not.

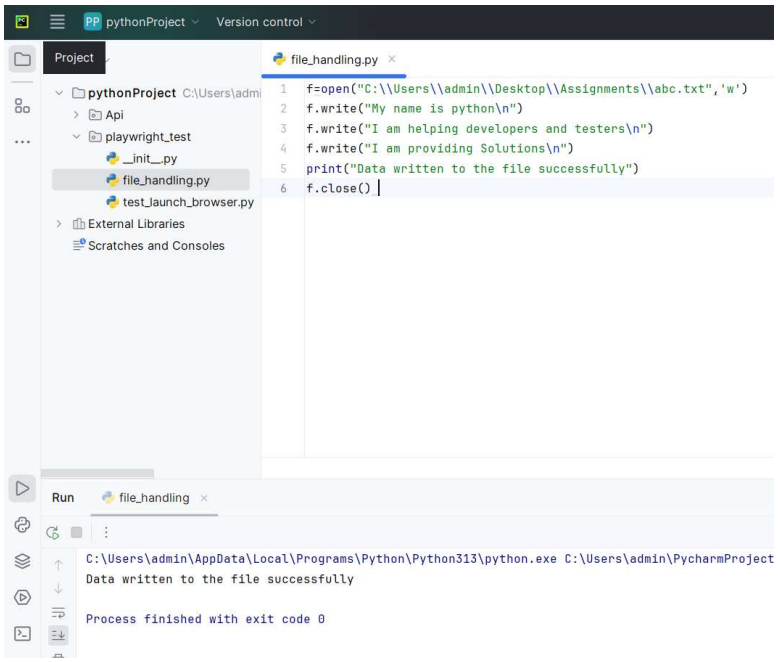


Writing Data to Text Files:

We can write character data to the text files by using the following 2 methods.

1) write(str)

2) writelines(list of lines)



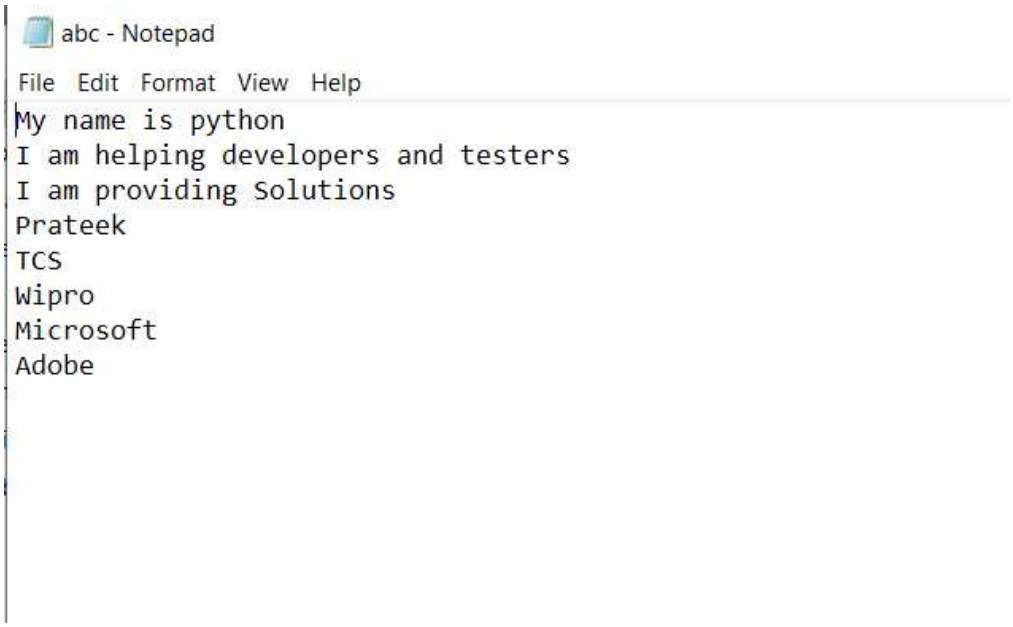

```
f=open("C:\\Users\\admin\\Desktop\\Assignments\\abc.txt",'w')
f.write("My name is python\n")
f.write("I am helping developers and testers\n")
f.write("I am providing Solutions\n")
print("Data written to the file successfully")
f.close()
```

Note -> if abc file not available then it will create automatically.

Now I want to append some data inside abc.txt file

```
f = open("abcd.txt","a")
```

```
f=open("C:\\Users\\admin\\Desktop\\Assignments\\abc.txt",'a')
list=["Prateek\n","TCS\n","Wipro\n","Microsoft\n", "Adobe"]
f.writelines(list)
```



Note: While writing data by using write() methods, compulsory we have to provide line separator(\n), otherwise total data should be written to a single line.

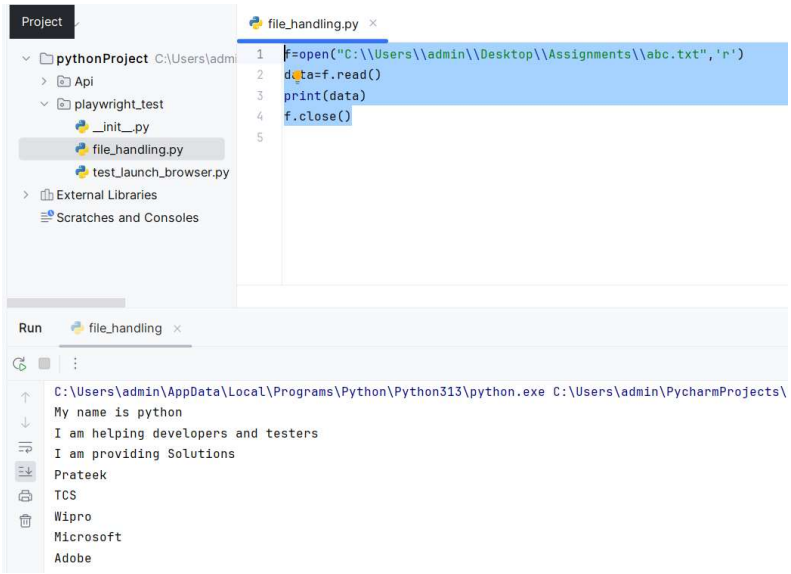
Reading Character Data from Text Files:

We can read character data from text file by using the following read methods.

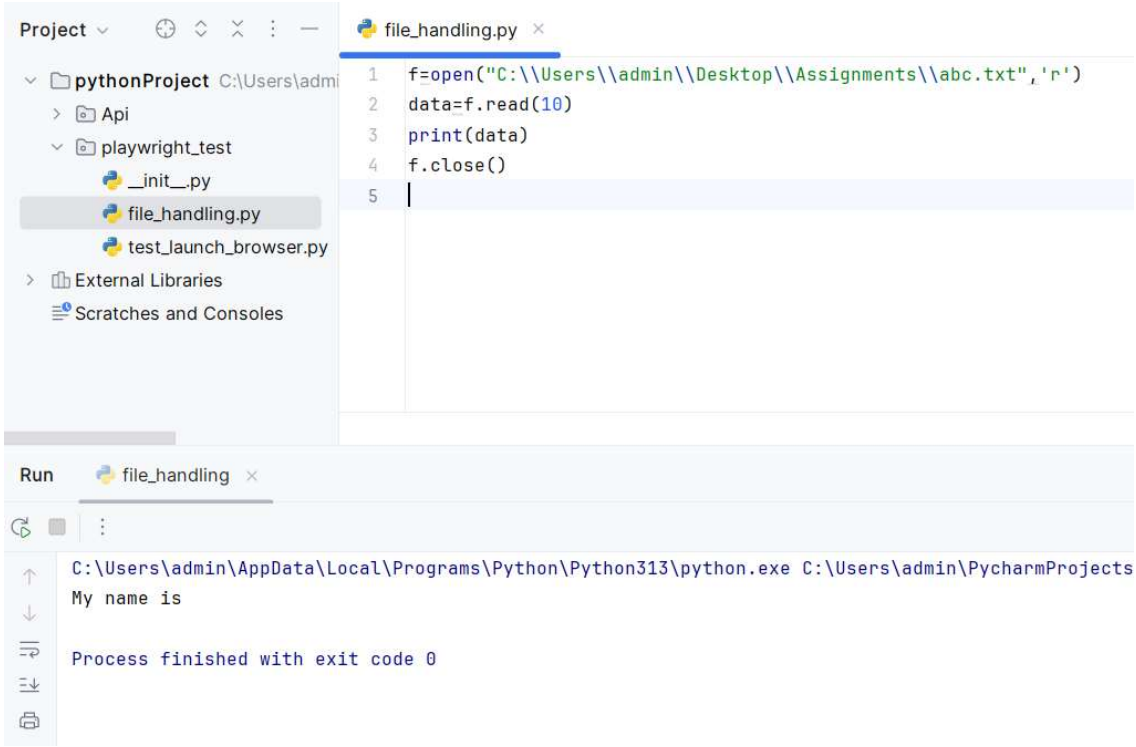
- read() → To read total data from the file
- read(n) → To read 'n' characters from the file
- readline() → To read only one line
- readlines() → To read all lines into a list

Data available till now inside abc.txt file is

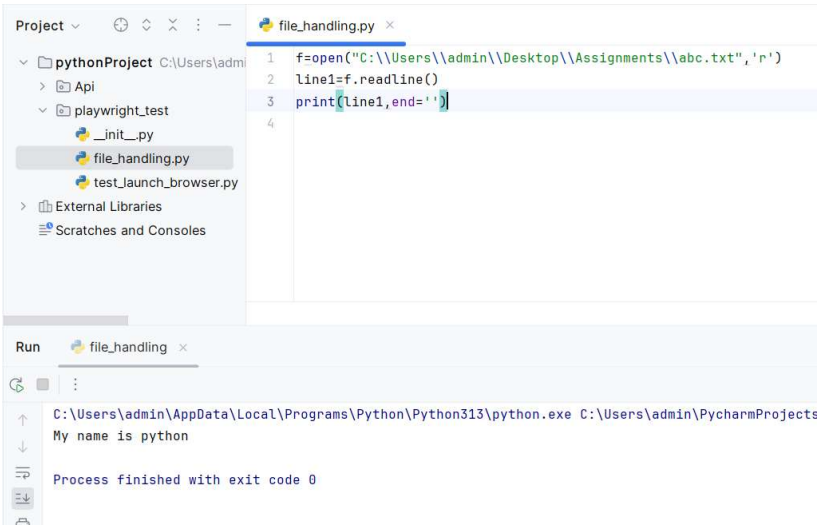
```
f=open("C:\\Users\\admin\\Desktop\\Assignments\\abc.txt",'r')
data=f.read()
print(data)
f.close()
```



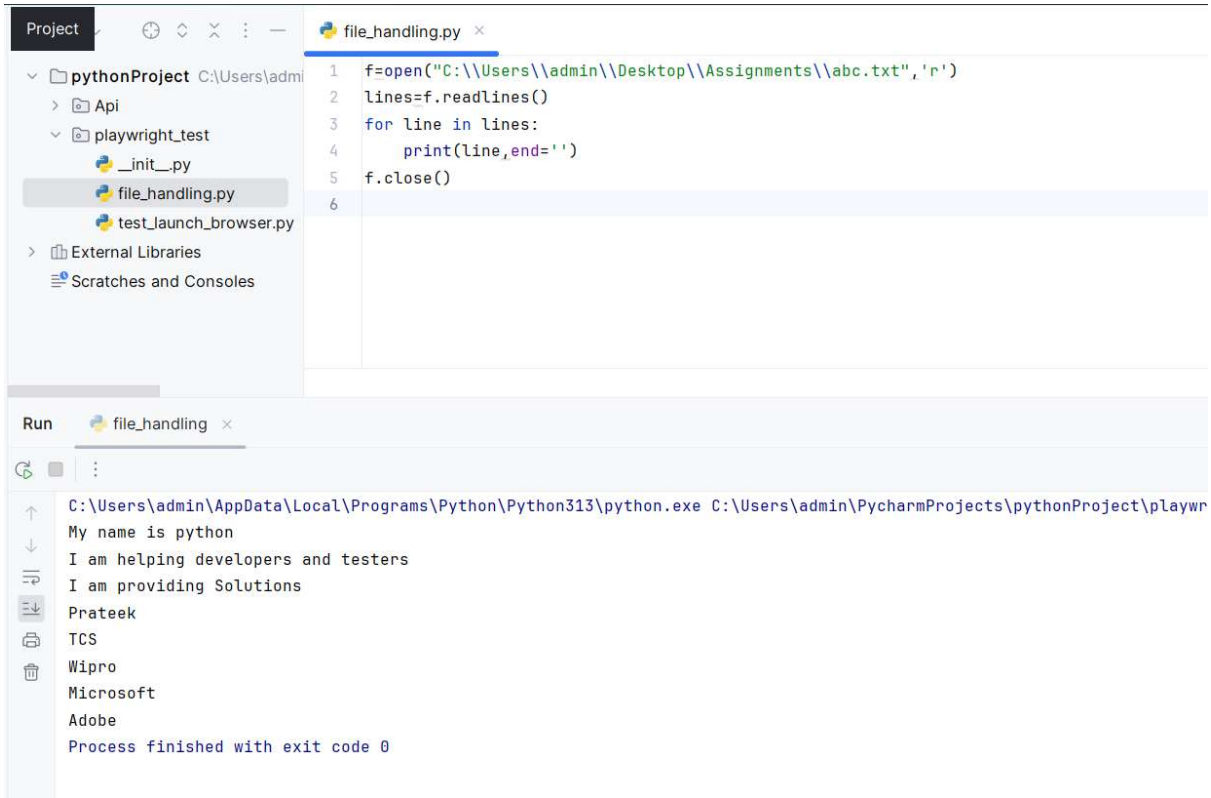

```
f=open("C:\\Users\\admin\\Desktop\\Assignments\\abc.txt",'r')
data=f.read(10)
print(data)
f.close()
```




```
f=open("C:\\Users\\admin\\Desktop\\Assignments\\abc.txt",'r')
line1=f.readline()
print(line1,end='')
```



```
f=open("C:\\Users\\admin\\Desktop\\Assignments\\abc.txt",'r')
lines=f.readlines()
for line in lines:
    print(line,end='')
f.close()
```

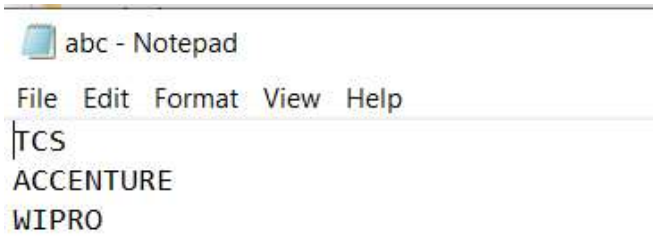


The with Statement:

- The with statement can be used while opening a file. We can use this to group file operation statements within a block.

- The advantage of with statement is it will take care closing of file, after completing all operations automatically even in the case of exceptions also, and we are not required to close explicitly.

```
with open("C:\\Users\\admin\\Desktop\\Assignments\\abc.txt","w") as f:
    f.write("TCS\n")
    f.write("ACCENTURE\n")
    f.write("WIPRO\n")
    f.close()
```



The seek() and tell() Methods:

tell():

- We can use tell() method to return current position of the cursor(file pointer) from beginning of the file. [can you plese telll current cursor position]
- The position(index) of first character in files is zero just like string index.

Seek():

We can use seek() method to move cursor (file pointer) to specified location.

Create a sample file

```
with open("example.txt", "w") as f:
    f.write("Learning Python is fun!")

# Read and manipulate the pointer
with open("example.txt", "r") as f:
    print(f"Initial position: {f.tell()}")          # Output: 0

    # Read the first 8 characters
    content = f.read(8)
    print(f"Read 8 chars: '{content}'")             # Output: 'Learning'
    print(f"New position: {f.tell()}")              # Output: 8

    # Jump to the 16th character
    f.seek(16)
    print(f"Moved to position: {f.tell()}")         # Output: 16

    # Read from the new position to the end
    remaining = f.read()
    print(f"Remaining text: '{remaining}'")         # Output: 'is fun!'
```


```python
# Create a sample file
with open("example.txt", "w") as f:
    f.write("Learning Python is fun!")

# Read and manipulate the pointer
with open("example.txt", "r") as f:
    print(f"Initial position: {f.tell()}")  # Output: 0

    # Read the first 8 characters
    content = f.read(8)
    print(f"Read 8 chars: '{content}'")     # Output: 'Learning'
    print(f"New position: {f.tell()}")      # Output: 8

    # Jump to the 16th character
    f.seek(16)
    print(f"Moved to position: {f.tell()}") # Output: 16

    # Read from the new position to the end
    remaining = f.read()
    print(f"Remaining text: '{remaining}'") # Output: 'is fun!'
```

Q1) To Know Current Working Directory

```
import os
cwd = os.getcwd()
print("Current Working Directory:",cwd)
```

How to check a particular File exists OR not?

We can use os library to get information about files in our computer.

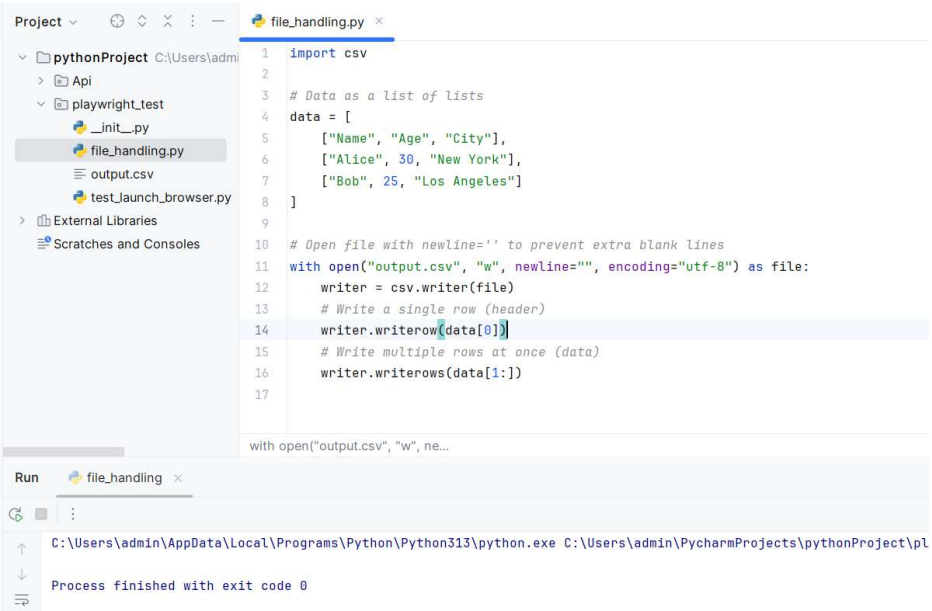
os module has path sub module,which contains isFile() function to check whether a particular file exists or not?

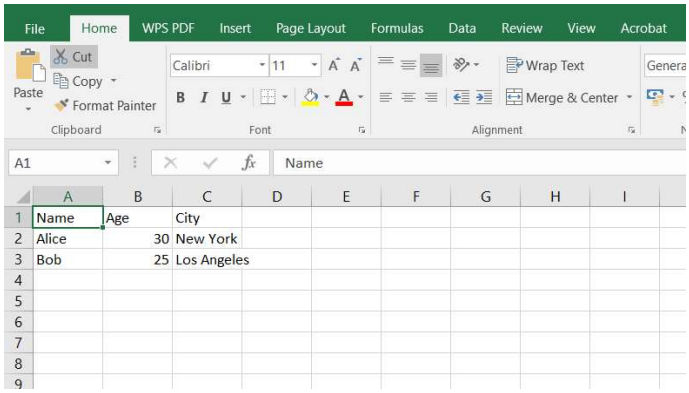
os.path.isfile(fname)

```
import os

# Define some paths to check
file_path = "example.txt"
dir_path = "my_folder"

# Scenario 1: Path is a regular file
if os.path.isfile(file_path):
    print(f"'{file_path}' is a valid file.")
else:
    print(f"'{file_path}' is NOT a file (it might be a directory or missing).")

# Scenario 2: Path is a directory (returns False)
if os.path.isfile(dir_path):
    print(f"'{dir_path}' is a file.")
else:
    print(f"'{dir_path}' is NOT a file (it is likely a directory).")
```

Handling CSV Files:

- CSV → Comma separated values
- As the part of programming, it is very common requirement to write and read data wrt csv files.
- Python provides csv module to handle csv files.
- `writerow()` : Writes a single row.
- `writerows()` : Writes an entire Iterable of rows.





Reading Data from CSV File:

python

```python
import csv

with open("emp.csv", "r") as f:
    r = csv.reader(f)
    for line in r:          # Reads one line at a time from disk
        for word in line:
            print(word, "\t", end='')
        print()             # Newline after each row
```

To Create Zip File:

We have to create ZipFile class object with name of the zip file, mode and constant ZIP_DEFLATED. This constant represents we are creating zip file.

f = ZipFile("files.zip","w","ZIP_DEFLATED")

Once we create ZipFile object,we can add files by using write() method.

f.write(filename)

```python
from zipfile import *
f=ZipFile("files.zip",'w',ZIP_DEFLATED)
f.write("file1.txt")
f.write("file2.txt")
f.write("file3.txt")
f.close()
print("files.zip file created successfully")
```

To perform unzip Operation:

We have to create ZipFile object as follows

```
f = ZipFile("files.zip","r",ZIP_STORED)
```

ZIP_STORED represents unzip operation. This is default value and hence we are not required to specify.

Once we created ZipFile object for unzip operation, we can get all file names present in that zip file by using namelist() method.

```
names  = f.namelist()
```

To Rename a Directory

```
import os
os.rename("mysub","newdir")
print("mysub directory renamed to newdir")
```

To Remove a Directory

```
import os
os.rmdir("mysub/mysub2")
print("mysub2 directory deleted")
```

write and read files in doc file

Install python-docx library →

Run command → pip install python-docx

Writing a New Document

```
from docx import Document

# Create a new document object
doc = Document()

# Add a title (Level 0) and a heading (Level 1)
doc.add_heading('My Python Document', 0)
doc.add_heading('Introduction', level=1)

# Add a paragraph
doc.add_paragraph('This file was created automatically using Python!')

# Add a paragraph with styled text (Runs)
p = doc.add_paragraph('Some text is ')
p.add_run('bold').bold = True
p.add_run(' and some is ')
p.add_run('italic.').italic = True

# Save the file
doc.save('demo.docx')
```


```
from docx import Document

# Create a new document object
doc = Document()

# Add a title (Level 0) and a heading (Level 1)
doc.add_heading('My Python Document', 0)
doc.add_heading('Introduction', level=1)

# Add a paragraph
doc.add_paragraph('This file was created automatically using Python!')

# Add a paragraph with styled text (Runs)
p = doc.add_paragraph('Some text is ')
p.add_run('bold').bold = True
p.add_run(' and some is ')
p.add_run('italic.').italic = True

# Save the file
doc.save('demo.docx')
```

Reading an Existing Document

```
from docx import Document

# Open the existing file
doc = Document('demo.docx')

# Iterate and print each paragraph's text
for i, para in enumerate(doc.paragraphs):
    print(f"Paragraph {i}: {para.text}")
```


```python
from docx import Document

# Open the existing file
doc = Document('demo.docx')

# Iterate and print each paragraph's text
for i, para in enumerate(doc.paragraphs):
    print(f"Paragraph {i}: {para.text}")
```